



Tribhuvan University
Faculty of Humanities and Social Sciences

A PROJECT REPORT
ON
FIRE DETECTION SYSTEM

Submitted to
Department of Computer Science and Application
Mechi Multiple Campus
Bhadrapur, Jhapa

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted By
Sudhir Dahal
6-2-2-98-2020
Santosh Bhandari
6-2-2-91-2020

Under the Supervision of
Mr. Raju Poudel
October 2025



Tribhuvan University

Faculty of Humanities and Social Sciences

Mechi Multiple Campus

Bhadrapur, Jhapa

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by **Sudhir Dahal and Santosh Bhandari** entitled “**Fire Detection System**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

SIGNATURE

Mr. Raju Poudel

PROJECT SUPERVISOR

Professor

BCA Department

Mechi Multiple Campus

Bhadrapur, Jhapa



Tribhuvan University

Faculty of Humanities and Social Sciences

Mechi Multiple Campus

Bhadrapur, Jhapa

LETTER OF APPROVAL

This is to certify that this project prepared by **Sudhir Dahal and Santosh Bhandari** entitled “**Fire Detection System**” in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Signature of Supervisor Mr. Raju Poudel Department of BCA Mechi Multiple Campus Bhadrapur, Jhapa	Signature of HOD/Coordinator Mr. Sunil Sharma Director Mechi Multiple Campus Bhadrapur, Jhapa
Signature of Internal Examiner _____	Signature of External Examiner _____

ACKNOWLEDGMENT

The project work presented in this report has been conducted and submitted at Mechi Multiple Campus, Faculty of Humanities and Social Sciences, Tribhuvan University, as a part of the Bachelor of Arts in Computer Application. This project has tested not only our technical skills but also our ability to work in a team and perform under various constraints. The journey of completing this project would not have been possible without the support and guidance of several individuals.

We are deeply thankful to our respected lecturers from the Department of Computer Application for their continuous support, guidance, and knowledge-sharing throughout the course of our studies. Their expertise greatly contributed to the successful development of this project.

We extend our heartfelt gratitude to our project supervisor, Mr. Raju Poudel, whose mentorship and feedback helped shape this project from idea to implementation. We are also thankful to our department coordinator, Mr. Sunil Sharma, for his administrative support and encouragement.

Lastly, we would like to thank our families, friends, and mentors for their unconditional love, motivation, and support throughout this journey. Their presence made this project not only possible but meaningful.

ABSTRACT

This project presents a Fire Detection System that operates in real-time to identify fire incidents using live video feeds from IP cameras. The system has been developed using Python, React.js, and computer vision techniques, integrating video streaming, fire detection algorithms, and an alert mechanism. The YOLOv8 AI model is implemented to detect flames with high accuracy under different conditions such as daylight, low light, and indoor environments.

Upon detecting fire, the system immediately generates alerts through visual notifications and sound alarms, ensuring quick response and minimizing potential damage. It also supports multiple camera inputs for monitoring larger areas like offices, factories, schools, and public places. This system aims to enhance fire safety infrastructure by providing an affordable, user-friendly, and efficient solution for early fire detection

Keywords: Fire Detection, YOLO, Computer Vision, Artificial Intelligence, Python, Real-Time Monitoring

Table of Contents

Supervisor's Recommendation	i
LETTER OF APPROVAL.....	ii
ACKNOWLEDGMENT	iii
ABSTRACT	iv
Table of Contents	v
List of Figures.....	vii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1: INTRODUCTION.....	1
1.1. Introduction.....	1
1.2. Problem Statement.....	1
1.3. Objectives.....	2
1.4. Scope and Limitation	2
1.4.1. Scope.....	2
1.4.2. Limitations.....	2
1.5. Development Methodology.....	3
1.6. Report Organization	3
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	4
2.1. Background Study	4
2.2. Literature Review	4
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	7
3.1. System Analysis.....	7
3.1.1. Requirement Analysis.....	7
i. Functional Requirements	7
ii. Non-Functional Requirements.....	8
3.1.2. Feasibility Analysis	9
i. Technical.....	9
ii. Operational.....	9
iii. Economic	9
iv. Schedule	9
3.1.3. Object Modelling using Class and Object Diagrams	10
3.1.4. Dynamic Modelling using State and Sequence Diagrams	11

3.1.5. Process Modelling using Activity Diagrams	12
3.2. System Design.....	13
3.2.1. Refinement of Class, Object, State, Sequence and Activity diagrams	13
3.2.2. Component Diagrams	15
3.2.3. Deployment Diagrams	15
3.3. Algorithm Details	16
CHAPTER 4: IMPLEMENTATION AND TESTING	18
4.1. Implementation	18
4.1.1. Tools Used.....	18
4.1.2. Implementation Details of Modules.....	18
4.2. Testing.....	19
4.2.1. Test Cases for Unit Testing	20
4.2.2. Test Cases for System Testing.....	21
4.3. Result Analysis	22
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION	23
5.1. Conclusion	23
5.2 Lesson Learnt/Outcomes.....	23
5.3 Future Recommendations	24
REFERENCES.....	25
APPENDIX.....	26

List of Figures

Figure 1: Iterative Waterfall Methodology	3
Figure 2: Use case Diagram.....	8
Figure 3: Gantt Chart	9
Figure 4: Object Modelling using Class Diagram	10
Figure 5: Object Modelling using Object Diagram.....	10
Figure 6: Dynamic Model using State diagram.....	11
Figure 7: Dynamic Model using Sequence diagram.....	11
Figure 8: Process Modelling using Activity Diagram.....	12
Figure 9: Refinement of Class Diagram.....	13
Figure 10: Refinement of Object Diagram	13
Figure 11: Refinement of State Diagram	14
Figure 12: Refinement of Sequence Diagram	14
Figure 13: Refinement of Component Diagram.....	15
Figure 14: Deployment Diagram	15

List of Tables

Table 1: Test Case for Video Streaming Module	20
Table 2: Test Case for Fire Detection Module	20
Table 3: Test Case for Alert and Notification Module	21
Table 4: Test Case for File Detection System.....	21

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CEO	Chief Executive Officer
CSS	Cascading Style Sheets
FDS	Fire Detection System
HTML	Hypertext Mark-up Language
IP	Internet Protocol
JS	JavaScript
RTSP	Real Time Streaming Protocol
URL	Uniform Resource Locator
YOLO	You Only Look Once

CHAPTER 1: INTRODUCTION

1.1. Introduction

Fire is one of the most dangerous hazards that can cause serious damage to life and property. Fires can start suddenly due to electrical faults, kitchen accidents, or natural causes like lightning. Once started, fire spreads very fast and can become uncontrollable if not detected early. Many buildings in Nepal and worldwide do not have adequate fire detection systems, making people and assets vulnerable. A Fire Detection System is a safety setup that quickly spots fire, smoke, or heat in a building. It uses devices like smoke detectors and alarms to warn people so they can escape and get help fast. These systems are used in places like homes, schools, offices, and hospitals to protect lives and property. Some can even trigger sprinklers or alert fire departments automatically. Early warning means more time to stay safe.

Traditional fire detection systems usually rely on smoke or heat sensors. While these systems are useful, they sometimes fail to detect fire early or give false alarms caused by cooking smoke or dust. Also, such systems cannot provide visual proof or location of the fire.

In this project, we aim to develop a Fire Detection System (FDS) that uses modern technology such as artificial intelligence (AI) and live video streaming. The system will use a camera to monitor a place continuously and an AI model called YOLOv8 to identify the presence of fire in the video feed in real-time. When fire is detected, the system will alert the users through sound or notifications immediately.

This system can help save lives and properties by detecting fire early, especially in homes, offices, schools, and public places.

1.2. Problem Statement

Fires can cause major losses if not detected on time. Traditional fire detection systems often fail because:

- They rely only on smoke or heat sensors which can give false alarms (e.g., from cooking smoke).
- They may not detect fire in open or large areas quickly.
- They cannot show exactly where the fire is happening.
- Many existing systems are expensive or complicated to install and maintain.

Hence, there is a need for a cost-effective, reliable, and accurate fire detection system that uses video analysis with AI for faster and better fire detection.

1.3. Objectives

The main objectives of this project are:

- To design and develop a fire detection system using live video streams.
- To train and implement the YOLOv8 AI model for detecting fire in real-time.
- To provide immediate alerts to users via buzzer or notification when fire is detected.
- To make the system easy to use, affordable, and suitable for small offices, schools, and homes.

1.4. Scope and Limitation

1.4.1. Scope

- Fire detection is powered by the YOLOv8 model, enabling accurate and fast identification of fire incidents in video frames.
- Alerts are triggered immediately through buzzers or notifications when fire is detected, ensuring quick response.
- Users can monitor the system through a web-based dashboard built with python for real-time status and alert viewing.
- Detection logs and incident data are stored in a database, allowing users to analyze past events and system performance.
- The system is designed to be cost-effective, using open-source tools and minimal hardware, suitable for homes, schools, and small offices.

1.4.2. Limitations

- The fire detection system requires a stable internet connection to stream video and send remote notifications, which may not be reliable in all areas.
- Detection accuracy may be affected by factors like low light, fog, dust, or camera angle, potentially causing missed or false alarms.
- The system is optimized for small enclosed areas performance may drop in large open spaces without additional cameras or sensors.
- False alarms can occur due to non-fire visuals like sunlight reflections, cooking smoke, or steam, depending on the environment.
- Real-time processing with YOLOv8 requires a system with adequate computing power (e.g., GPU or high-performance CPU).

1.5. Development Methodology

The Fire Detection System was developed using the Iterative Waterfall Model. This model follows a step-by-step process including requirement analysis, system design, implementation, and testing. After each phase, feedback is reviewed and necessary improvements are made before moving to the next stage. It provided a clear and organized structure, which was ideal for this project since the requirements were well-defined and the tools used were stable.

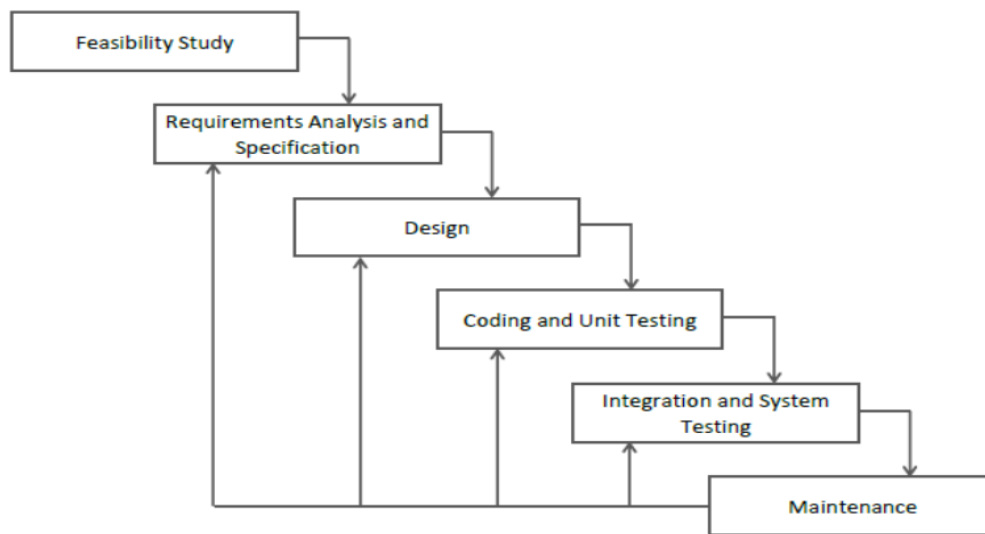


Figure 1: Iterative Waterfall Methodology

1.6. Report Organization

This project is divided into five major chapters.

Chapter 1 of the project gives the background information on the project and it also tells about the objectives, scope and limitation of the project.

Chapter 2 of the project is Background Study and Literature Review. This chapter provides the background information about topic and provides the known information and theories.

Chapter 3 of the project is System Analysis and Design. In this chapter, system will be analyzed by performing the requirement and feasibility analysis. This chapter also provides the system designs of the project.

Chapter 4 presents the implementation of the project, the tools used, and details of unit and system testing with result analysis.

Chapter 5 of the project is Conclusion and Future Recommendation. It is the last chapter which provides conclusion, outcomes and further recommendations related to the project

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1. Background Study

Fire detection systems play a crucial role in protecting people and property from fire hazards. Traditional fire alarms mostly rely on sensors that detect smoke or heat, but these can sometimes trigger false alarms due to cooking smoke, steam, or dust. Also, they might not detect fire quickly in large or open spaces.

Recent advances in technology have led to smarter fire detection methods using cameras and artificial intelligence (AI). AI models, like YOLOv8, can analyze live video feeds to identify fire or smoke in real time. This allows for quicker detection and more accurate alerts compared to traditional sensors.

The system uses video streaming protocols such as RTSP (Real-Time Streaming Protocol) to get continuous video from cameras. These live feeds are processed by AI models to monitor fire-prone areas constantly. When fire is detected, the system can immediately alert users through alarms or notifications. Integrating AI with video surveillance improves fire detection by reducing false alarms and providing visual confirmation of the fire location. However, challenges such as environmental factors (fog, reflections), limited data for training AI, and computational needs still exist.

Despite these challenges, AI-based fire detection systems are becoming more affordable and easier to install, making them a promising solution for homes, offices, schools, and factories. They enhance safety by providing early warnings, helping people evacuate on time and reducing property damage.

2.2. Literature Review

Fire detection systems are critical in providing early warnings to prevent damage and save lives. Traditionally, these systems relied on physical sensors like smoke detectors and heat sensors, which detect signs of fire through changes in the environment. However, these sensor-based systems sometimes face issues such as false alarms caused by dust, steam, or other environmental factors, leading to delayed or inaccurate detection. To address these challenges, recent advances have focused on video-based fire detection methods that use cameras to monitor fire characteristics such as flame movement, color, and smoke patterns. Real-Time Streaming Protocol (RTSP) allows for continuous video streaming from surveillance cameras, enabling real-time monitoring. By integrating RTSP with deep learning algorithms like YOLOv8, fire

detection systems can analyze video streams quickly and accurately, improving detection performance and reducing false alarms.

Fire detection systems are essential safety technologies designed to identify fires as early as possible, thereby minimizing harm to lives and property. These systems have evolved from simple smoke or heat sensors to complex solutions that combine multiple detection methods for improved accuracy and speed [1].

Traditional fire detection methods typically use physical sensors such as smoke detectors or heat detectors; however, these sensors can be affected by environmental conditions like dust, steam, or high humidity, which may lead to false alarms or delayed detection [2]

A comprehensive study of fire behavior, which is fundamental for designing effective detection systems. Fire properties such as temperature, flame characteristics, and smoke production evolve during different stages of a fire. This knowledge helps in selecting and positioning sensors to detect fire as early as possible. Fire detection systems work in conjunction with suppression technologies like sprinklers to enhance overall building safety. [3]

RTSP ensures that video feeds are transmitted efficiently and reliably, even over networks with variable bandwidth, which is critical for timely fire detection. Advanced computer vision techniques, particularly those based on deep learning models like YOLOv11 (You Only Look Once version 11), have been integrated with RTSP streams. YOLOv11 processes the video frames quickly to identify and localize fire instances with high precision and low latency, making it highly effective in complex environments with varying lighting and background conditions [4].

A fire detection system using convolutional neural networks (CNNs) to analyze video frames in real time. Their approach significantly improves detection speed and accuracy compared to traditional sensor-based systems. The system effectively differentiates real fires from visual artifacts like reflections, a common challenge in video analysis. Tests conducted in both indoor industrial environments and outdoor forest areas demonstrate the model's versatility and reliability. They also explore the potential for integrating such AI-driven detection systems into broader smart city frameworks, which could revolutionize urban fire safety. [5]

An authoritative source offering detailed guidelines for fire detection system installation, testing, and maintenance. It highlights innovations such as multi-criteria detectors that combine inputs from smoke, heat, and carbon monoxide sensors to reduce

false alarms. The site provides practical training and compliance information for fire safety professionals and facility managers. The NFPA regularly updates its standards based on new research and incident data, ensuring that fire detection systems meet the latest safety requirements. This ongoing effort supports effective risk management and regulatory compliance worldwide. [6]

In conclusion, the evolution of fire detection systems from traditional sensor-based methods to advanced video and AI-driven technologies has significantly enhanced their effectiveness. The use of RTSP for real-time video streaming combined with powerful detection models like YOLOv8 enables rapid and precise fire identification. These developments, along with IoT integration for timely alerts, make modern fire detection systems highly efficient and responsive. Continuous research and technological improvements are expected to further increase the accuracy, scalability, and accessibility of fire detection solutions, contributing to safer environments across various applications.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1. System Analysis

It involves a detailed examination of the project's goals, the problems it aims to solve, and the functional and non-functional requirements needed to develop the system effectively. In this section, we analyze the existing issues with traditional fire detection methods, define user and system requirements, and assess the feasibility of the proposed solution

3.1.1. Requirement Analysis

The requirement analysis phase defines the essential functions and qualities the system must possess to meet the users' needs effectively. It is divided into two main categories. The first one is Functional Requirements and second is Non-Functional Requirements.

i. Functional Requirements

Functional requirements describe the core operations and behaviors the system must perform. These define what the system should do.

Manage Cameras

The user can add, update, delete, and view surveillance cameras connected to the fire detection system. This allows the user to configure camera locations, monitor real-time video feeds, and ensure the entire area is covered effectively for fire surveillance.

Manage Alerts

The user can view all alerts generated by the system when fire or smoke is detected. The user is able to acknowledge, respond to, or mark alerts as resolved. This ensures timely action is taken in case of emergencies and helps maintain a record of alert responses.

Manage Detection History

The user can access and review a history of fire detection events. This includes the date, time, and nature of each incident. The user can filter or search through these records to analyse past incidents and system performance.

Receive Notifications

The user receives real-time notifications when the system detects fire or smoke. Notifications are displayed on the user interface, enabling immediate awareness and response to emergencies.

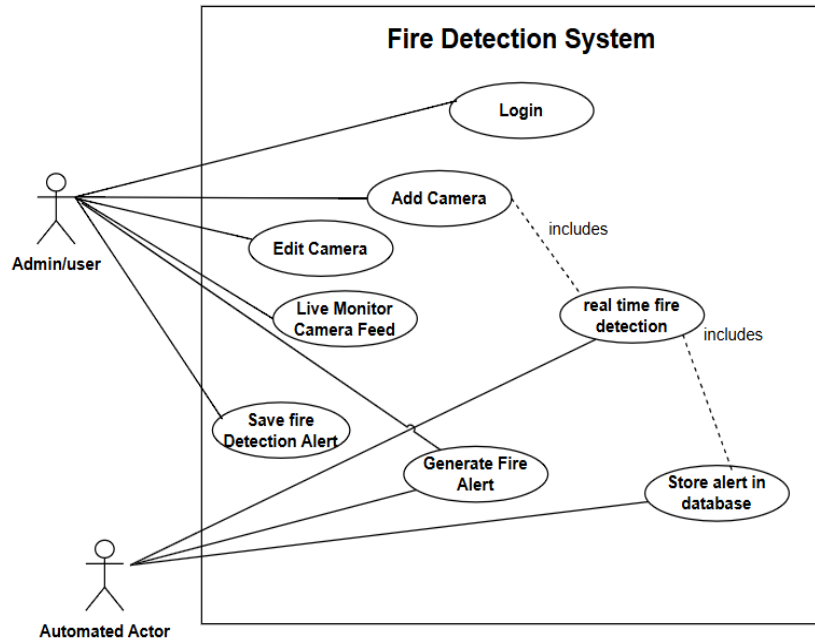


Figure 2: Use case Diagram

ii. Non-Functional Requirements

These describe how the system should perform, focusing on quality, usability, and operational constraints.

➤ **Performance:**

The system must detect fire quickly with minimal delay, ideally under one second. YOLOv8 should maintain high accuracy during real-time video analysis to ensure early warnings.

➤ **Scalability:**

The system should support multiple camera streams and scale easily for use in homes, schools, or offices without major changes to the architecture.

➤ **Reliability:**

It must run continuously and handle interruptions like power loss or network issues. Automatic recovery should be built in to keep the system running smoothly.

➤ **Usability:**

The dashboard should be simple and easy to use, allowing users to view live feeds, alerts, and logs without needing technical skills.

➤ **Safe and Secure:**

➤ Only trusted users can access the system (if there is an online panel).

3.1.2. Feasibility Analysis

i. Technical

The system is technically feasible using tools like YOLOv8 from Ultralytics, Roboflow for dataset preparation, and OpenCV for video processing. These open-source tools are reliable, well-documented, and compatible with standard hardware, making development and deployment straightforward.

ii. Operational

The system is easy to use and can be operated with minimal training. The web dashboard allows users to monitor live feeds and receive alerts without needing technical expertise, making it suitable for everyday use in homes, schools, and offices.

iii. Economic

The project is cost-effective, using free and open-source software and affordable hardware. No expensive licenses or specialized equipment are required, making the system financially practical for small-scale installations.

iv. Schedule

The system can be developed within a short time frame since the requirements are clear and tools like YOLOv8 and Roboflow speed up training and implementation. The full system can be completed in a few weeks depending on the scope.

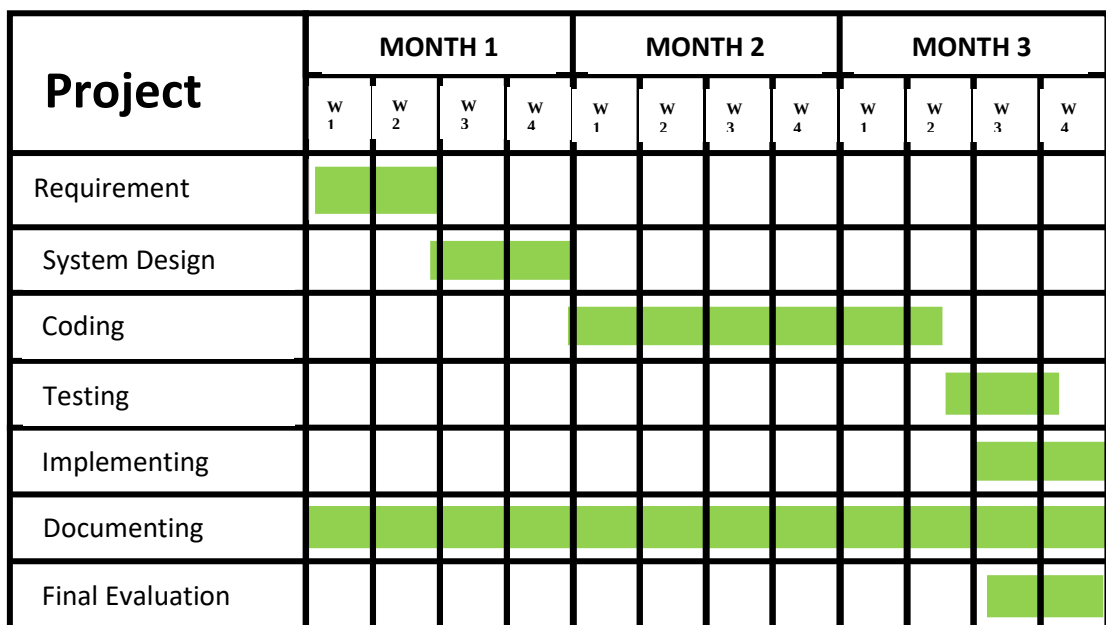


Figure 3: Gantt Chart

3.1.3. Object Modelling using Class and Object Diagrams

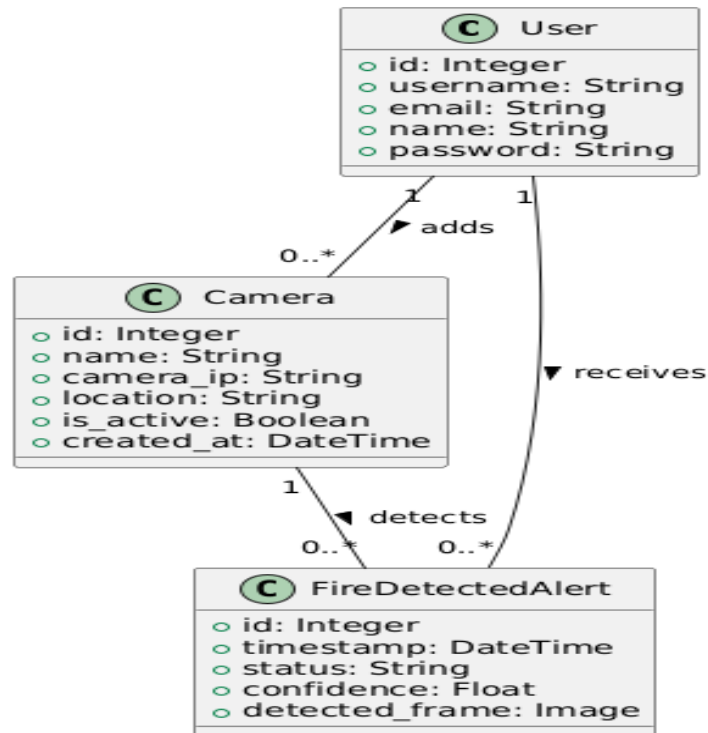


Figure 4: Object Modelling using Class Diagram

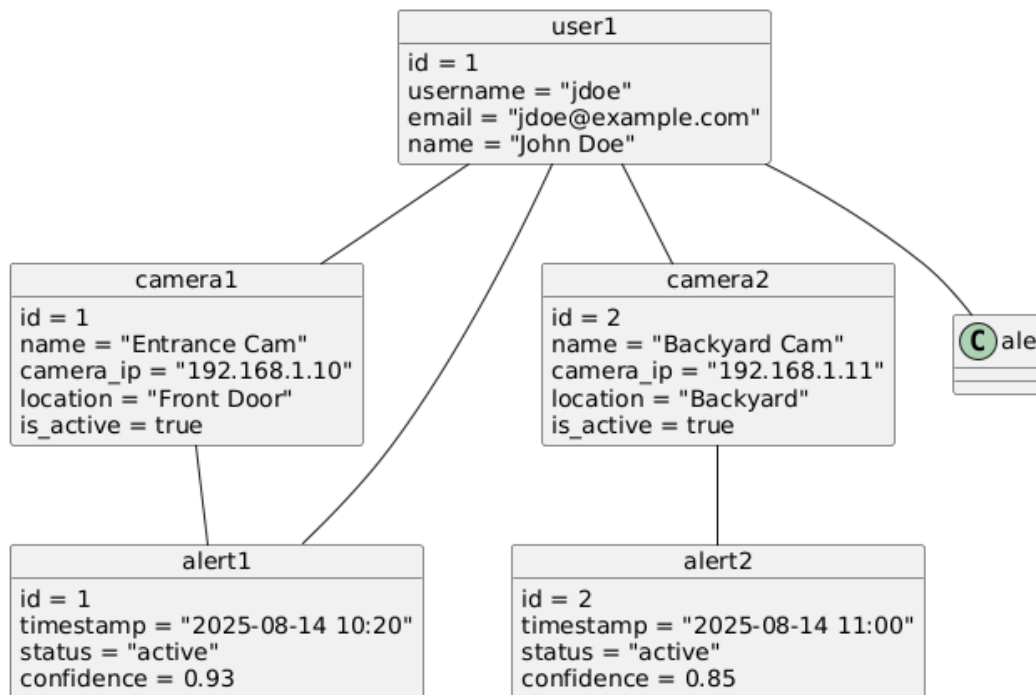


Figure 5: Object Modelling using Object Diagram

3.1.4. Dynamic Modelling using State and Sequence Diagrams

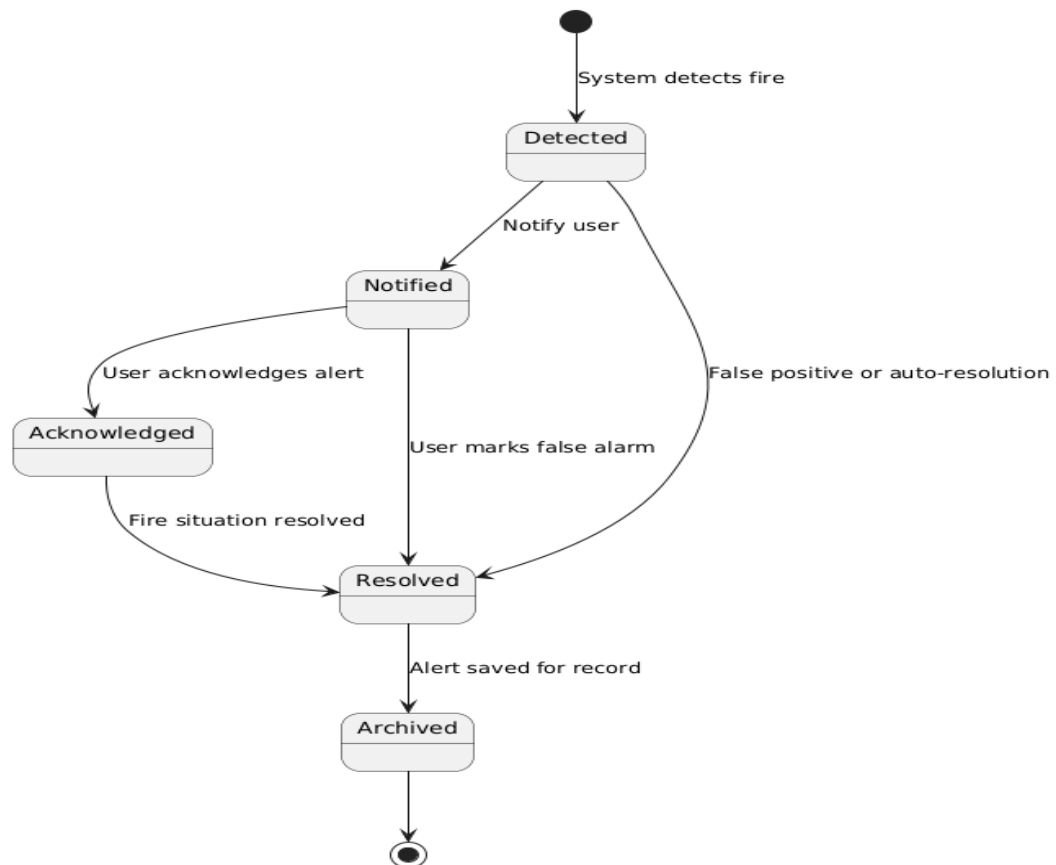


Figure 6: Dynamic Model using State diagram

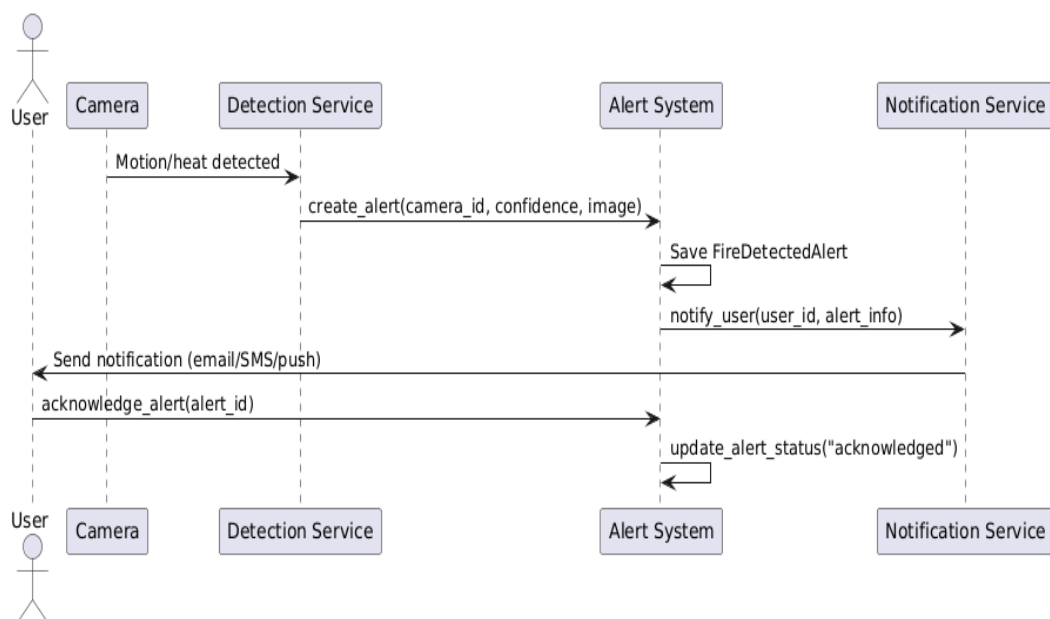


Figure 7: Dynamic Model using Sequence diagram

3.1.5. Process Modelling using Activity Diagrams

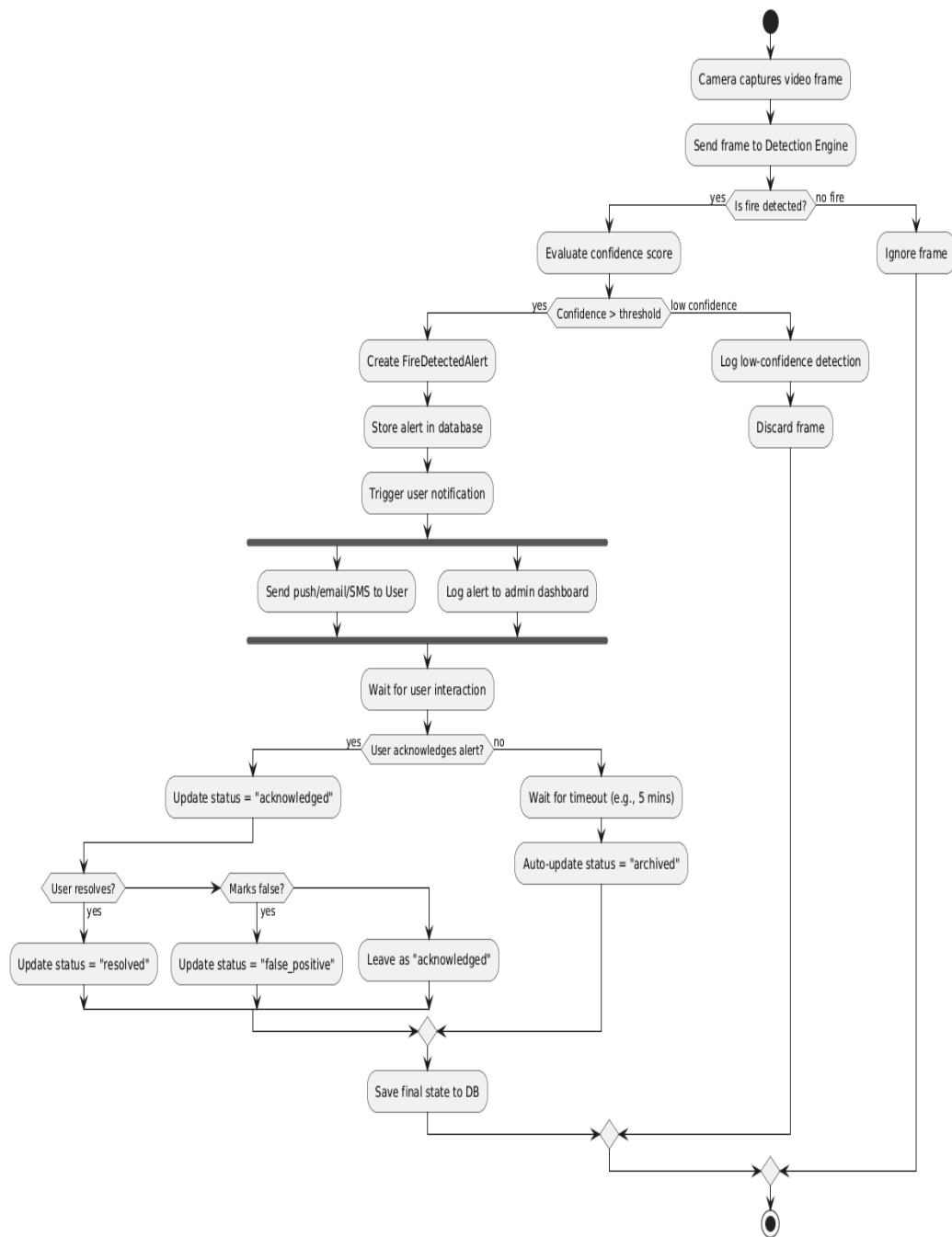


Figure 8: Process Modelling using Activity Diagram

3.2. System Design

3.2.1. Refinement of Class, Object, State, Sequence and Activity diagrams

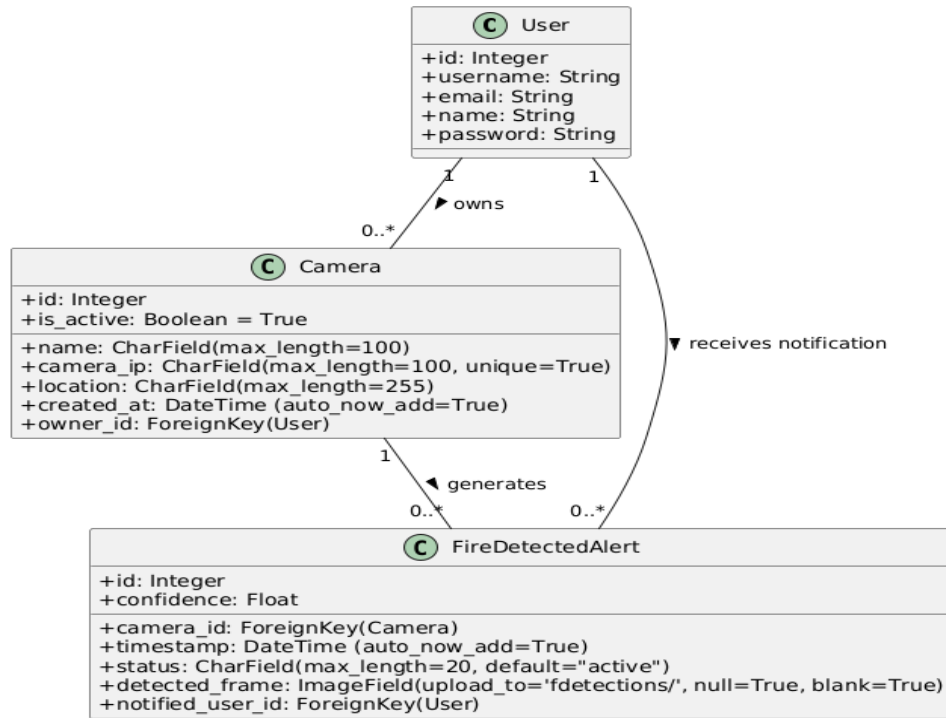


Figure 9: Refinement of Class Diagram

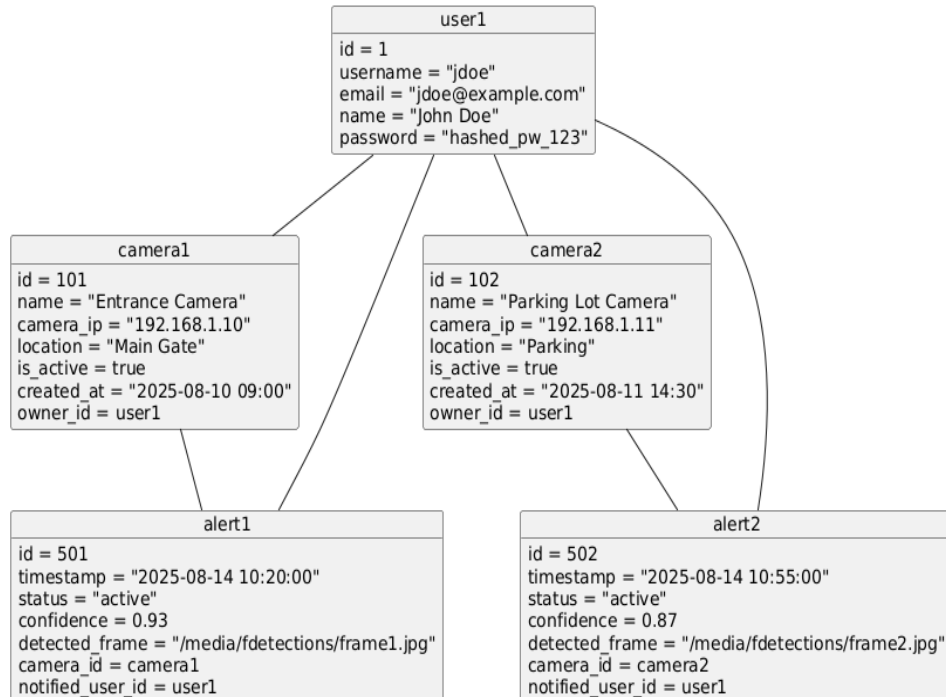


Figure 10: Refinement of Object Diagram

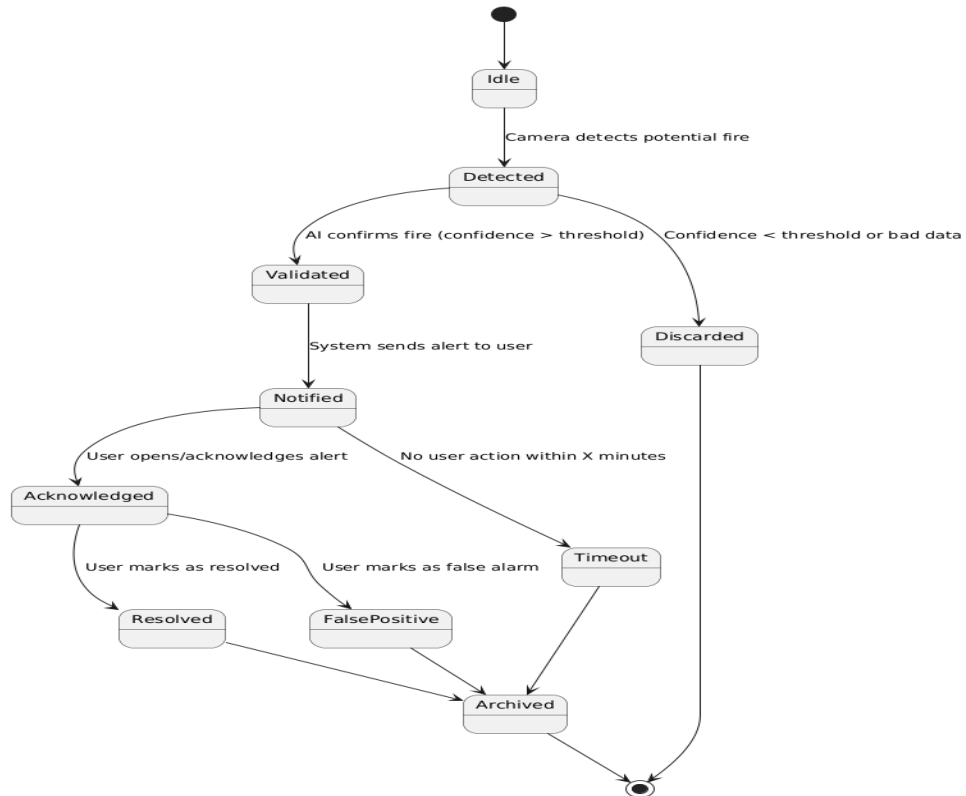


Figure 11: Refinement of State Diagram

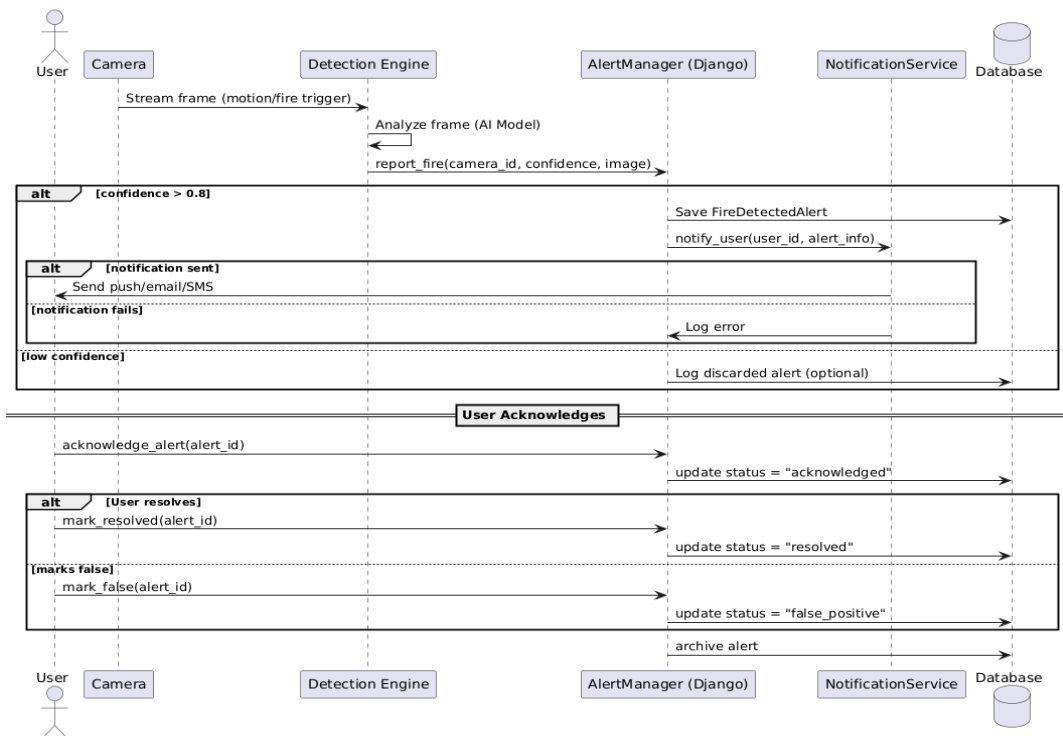


Figure 12: Refinement of Sequence Diagram

3.2.2. Component Diagrams

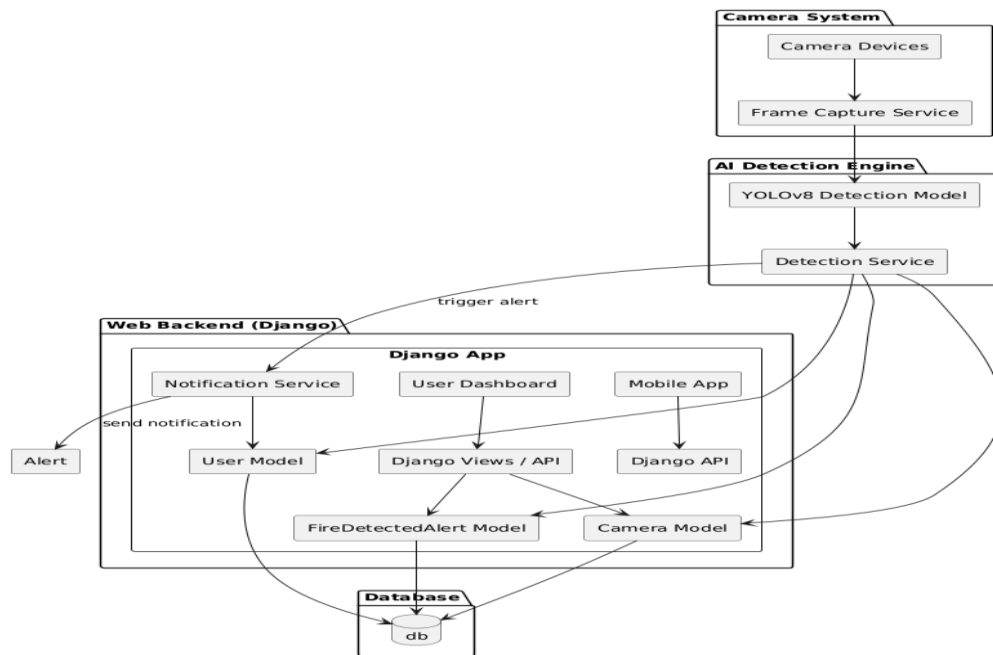


Figure 13: Refinement of Component Diagram

3.2.3. Deployment Diagrams

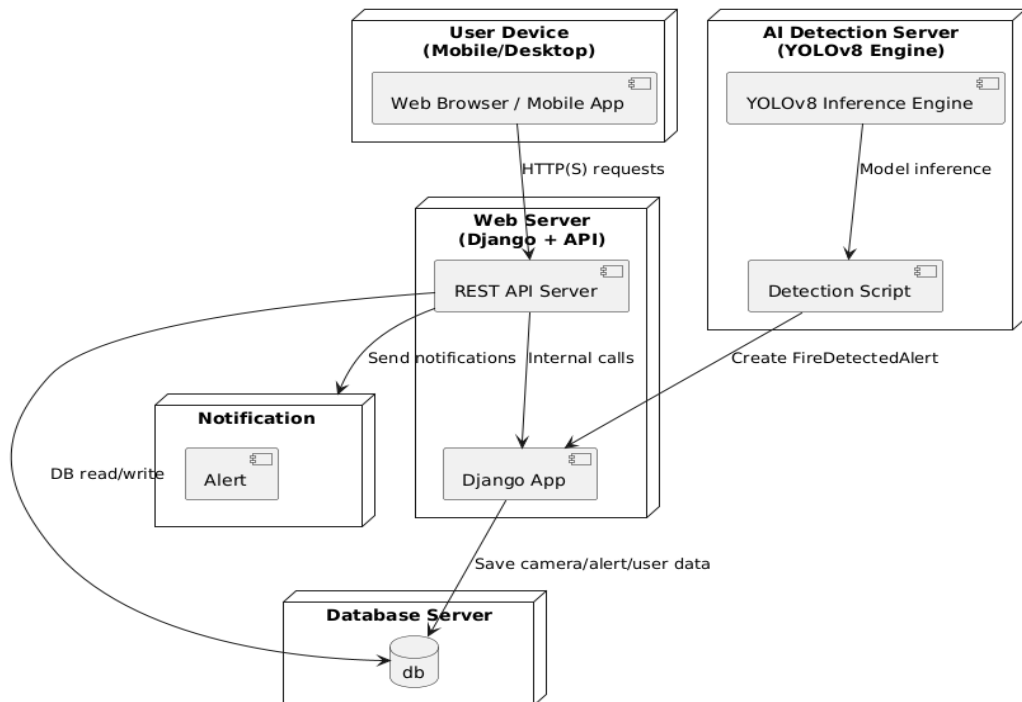


Figure 14: Deployment Diagram

3.3. Algorithm Details

Algorithm for Fire Detection Using YOLOv8

YOLOv8 (You Only Look Once, Version 8) is an advanced real-time object detection algorithm developed by Ultralytics. It is a single-stage, anchor-free object detector that uses a Convolutional Neural Network (CNN) to detect and localize objects in images or video frames in one forward pass.

In this Fire Detection System, YOLOv8 is employed to detect fire in camera frames, including live video streams, and send fire alert data, such as confidence, camera IP, and detected frame to the system

How YOLOv8 Works

Input Image Preprocessing: Input frame from a live camera (or video stream) is resized to a fixed shape, e.g., 640×640 pixels. Pixel values are normalized to the range [0,1]. Using the formula:

$$X_{\text{norm}} = X/255$$

Where, X = original image pixel values.

In digital images, pixel values usually range from 0 to 255 for each color channel (Red, Green, Blue).

Xnorm = normalized pixel values, which are scaled to range [0,1]

Why We Normalize

Stability: Neural networks work better with inputs in a smaller, consistent range.

Faster convergence: Helps the model train faster.

Avoids large values: Prevents numerical issues in computation.

Backbone in YOLOv8

The backbone is a Convolutional Neural Network (CNN) inside YOLOv8. Common backbones: CSPDarknet, Efficient Net, or custom lightweight CNNs. Its main role is to convert the input image into multi-scale feature maps.

Mathematical Representation:

$$F = f_{\text{backbone}}(X_{\text{norm}})$$

Where:

X_{norm} = normalized image input

f_{backbone} = CNN backbone network

F = extracted feature maps

How It works

Convolution Layer: Detects local patterns like edges, corners, or textures. Output: multiple feature maps representing different filters.

Activation Function: Introduces non-linearity to capture complex patterns like irregular fire shapes.

Pooling Layer (Optional): Reduces spatial size, keeps the most important information, and reduces computation.

Stacking Multiple Layers: Multiple convolution layers extract hierarchical features: Early layers like basic features (edges, colours) and Deeper layers like complex features (flame shapes, smoke patterns)

Feature Fusion (Neck):

$F_{fused} = f_{neck}(F1, F2, F3)$

Where:

$F1, F2, F3$ = feature maps from different stages of backbone (high, medium, low resolution)

f_{neck} = Feature Pyramid Network (FPN) or Path Aggregation Network (PAN)

F_{fused} = fused feature maps combining all scales.

E.g.: Suppose a camera captures a frame showing a fire in a room. The backbone:

-It Detects bright orange and red flame patterns.

It recognizes irregular shapes typical of fire.

-It preserves spatial information (where the fire is in the frame).

In live video streaming, every frame captured from an IP camera must be processed in real-time. YOLOv8 relies on its Backbone CNN to extract meaningful features from each frame, such as Flame colours (orange/red regions), Smoke textures, Shapes and patterns indicative of fire. And Use OpenCV or a similar library to capture frames from live IP camera and same process follows Input Image Pre-processing

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1. Implementation

The Fire Detection System was implemented to monitor live video feeds from cameras, detect fire in real-time using AI, and send immediate alerts. The system is designed to run on a local server and allows users to monitor multiple cameras simultaneously through a web-based dashboard. The implementation focused on integrating video streaming, fire detection, alert notifications, and database logging for event history. The development followed modular design principles, making the system scalable, efficient, and user-friendly.

4.1.1. Tools Used

The backend of the Fire Detection System was developed using Python, which provides powerful libraries for video processing and AI integration. Django was used to create a RESTful API for managing cameras, alerts, and detection history. Django's built-in authentication and validation ensured that only authorized users could access the system securely. OpenCV was utilized to capture and process live video streams from webcams and IP cameras, while the YOLOv8 AI model from Ultralytics performed real-time fire detection with high accuracy. To handle multiple camera streams simultaneously, Python's threading and concurrency tools were used, ensuring smooth performance. For the frontend, React.js was employed to build a dynamic and interactive web dashboard where users can view live camera feeds, alerts, and detection history. The dashboard uses JavaScript (JSX) for logic and interaction, and Tailwind CSS was applied to create a clean and responsive design. HTML serves as the base structure, while Vite was used as the development server and build tool to efficiently manage the frontend code.

The system stores data such as camera information and alert logs in a database. Either SQLite can be used to save this structured data and provide fast retrieval for real-time display

4.1.2. Implementation Details of Modules

1. Video Streaming Module

This module captures live video from IP cameras or webcams using RTSP or OpenCV. It continuously fetches video frames and sends them to the detection system. The video

can also be viewed live through a web dashboard. It ensures smooth, real-time streaming for fire monitoring.

2. Fire Detection Module

This is the main part of the system that detects fire using the YOLOv8 AI model. It analyzes each video frame in real-time and identifies fire if present. The model is trained on a fire dataset to improve accuracy. Once fire is detected, it sends the result to the alert system.

3. Alert and Notification Module

When fire is detected, this module sends alerts through sound, email, or on-screen messages. It helps people respond quickly to danger. It may also include a buzzer or future SMS support. This ensures the warning reaches users without delay.

4. Dashboard Module

The dashboard allows users to view live streams, receive alerts, and manage the system. It shows detection history and camera feeds in one place. Built using web technologies, it is user-friendly and can be accessed on browsers. It makes the system easy to use.

5. Camera Management Module

This module helps users add, update, or delete camera streams. It stores camera info like name and stream URL. Useful for monitoring multiple locations from one system. It ensures flexibility and scalability.

6. Database and Logging Module

All fire detections and events are stored in a database. It keeps records like time, location, and detection image. This helps review past incidents and improve system performance. Logs can also be used for reports and analysis.

4.2. Testing

Testing is a crucial part of the Fire Detection System to ensure its accuracy, reliability, and responsiveness. The system was tested in different scenarios using live video streams to evaluate its performance in detecting fire. Both unit testing and system testing were performed to verify that all modules work as expected and the integrated system functions correctly. Testing helped identify potential issues, improve accuracy, and ensure that alerts are sent immediately when fire is detected.

4.2.1. Test Cases for Unit Testing

Table 1: Test Case for Video Streaming Module

Project Name: File Detection System					
Test Case ID: TC1					
Module Name: Video Streaming Module					
Test Designed /Executed by: Santosh Bhandari and Sudhir Dahal					
Description: Testing live video streaming from IP cameras for valid and invalid scenarios					
S. N	Test Case Description	Input	Expected Result	Actual Result	Status
1	Test valid video stream	url=http://127.0.0.1:5000/main	Video stream displayed correctly	Video displayed correctly	Pass
2	Test invalid URL	http://127.0.0.1:5000/invalid	Video not displayed / error	Error displayed	Pass
3	Camera disconnected	RTSP or HTTP stream disconnected	Error message / no video feed	Error displayed	Pass
4	Test multiple streams	http://127.0.0.1:5000/main, http://127.0.0.1:5000/test	All streams display simultaneously	Streams displayed correctly	Pass

Table 2: Test Case for Fire Detection Module

Project Name: File Detection System					
Test Case ID: TC2					
Module Name: Fire Detection Module					
Test Designed /Executed by: Santosh Bhandari and Sudhir Dahal					
Description: Testing fire detection accuracy in different scenarios					
S. N	Test Case Description	Input	Expected Result	Actual Result	Status
1	Detect fire in daytime	Video with fire in daylight	Fire detected	Fire detected	Pass
2	Detect fire in low light	Video with fire at night / low light	Fire detected	Fire detected	Pass
3	Test non-fire objects	No fire detected	No fire detected	Error displayed	Pass
4	Continuous detection	Long video feed with fire	Fire detected continuously	Fire detected continuously	Pass

Table 3: Test Case for Alert and Notification Module

Project Name: File Detection System					
Test Case ID: TC3					
Module Name: Alert and Notification Module					
Test Designed /Executed by: Santosh Bhandari and Sudhir Dahal					
Description: Testing alert generation hen fire is detected.					
S. N	Test Case Description	Input	Expected Result	Actual Result	Status
1	Sound alert	Fire detected	Buzzer sounds	Buzzer sounds	Pass
2	On-screen notification	Fire detected	Notification displayed	Notification displayed	Pass
3	Multiple alerts	Fire detected in multiple cameras	All alerts triggered	All alerts triggered	Pass

4.2.2. Test Cases for System Testing**Table 4: Test Case for File Detection System**

Project Name: File Detection System					
Test Case ID: TC4					
Module Name: Alert and Notification Module					
Test Designed /Executed by: Santosh Bhandari and Sudhir Dahal					
Description: Testing overall system performance					
S. N	Test Case Description	Input	Expected Result	Actual Result	Status
1	Real-time fire detection	Live video with fire	Fire detected immediately	Fire detected immediately	Pass
2	Stream fire.mp4 from single camera	Camera URL: http://127.0.0.1:5000/test	Live stream displays frames from fire.mp4 without lag	Stream displayed smoothly	Pass
3	Multiple camera feeds	2–3 cameras active	System handles all feeds correctly	System handles all feeds correctly	Pass
4	Notification delivery	Fire detected	Admin receives alerts instantly	Admin receives alerts instantly	Pass
5	Module integration	Full system	All modules work together	All modules work together	Pass

6	False alarm testing	Non-fire videos	No false alert	No false alert	Pass
7	Performance under load	4 simultaneous camera streams	All streams smooth, CPU under 85%	Streams stable, CPU ~82%	Pass

4.3. Result Analysis

The Fire Detection System successfully detected fire in real-time during all tested scenarios with high accuracy. The YOLOv8 model demonstrated effective recognition of fire patterns in different lighting conditions and environments. Alerts were sent immediately to Admin users, allowing quick response. The system-maintained logs accurately, providing a useful record of incidents for analysis. Minor false alarms occurred due to non-fire visuals like sunlight reflections or smoke from cooking, which can be reduced in future updates by improving AI training. Overall, the system is reliable, efficient, and suitable for small offices, schools, and homes, fulfilling the objectives of the project

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION

5.1. Conclusion

The Fire Detection System developed in this project successfully detects fire in real-time using live video streams and AI-based analysis with YOLOv8. It provides immediate alerts to Admin users through sound or notifications, helping to minimize damage to life and property. The system maintains logs of all detection events for review and monitoring, making it easier to track incidents. By combining continuous camera monitoring with automated alerts, it ensures faster response than traditional sensor-based systems. The web-based dashboard allows Admin users to manage cameras, view alerts, and access detection history in a simple and user-friendly way. Overall, the system is reliable, affordable, and suitable for homes, schools, and small offices.

The performance of the system shows the benefits of integrating AI and real-time video analysis for safety. Adding role-based features for Admin users, such as managing cameras, controlling alerts, and accessing logs securely, improves accountability and management. Future improvements like mobile app support, cloud-based storage, scalability for multiple buildings, and energy-efficient operation can make the system more robust. Integration with IoT devices, such as smoke detectors and sprinklers, will allow fully automated responses. Continuous AI model updates will help reduce false alarms and improve accuracy. These enhancements will make the system more intelligent, reliable, and effective for diverse environments, ensuring better protection for people and property.

5.2 Lesson Learnt/Outcomes

The outcome of the project is a working system that can detect fires in real time, send alerts, and keep records for analysis. A feature of Immediate alert mechanism using a buzzer and notification system. It can be applied in homes, offices, and industries, and can be upgraded with more features in the future. I learned the importance of designing a user-friendly interface so users can quickly understand alerts and take action. This project improved my skills in web development, API integration, and problem-solving for safety systems.

5.3 Future Recommendations

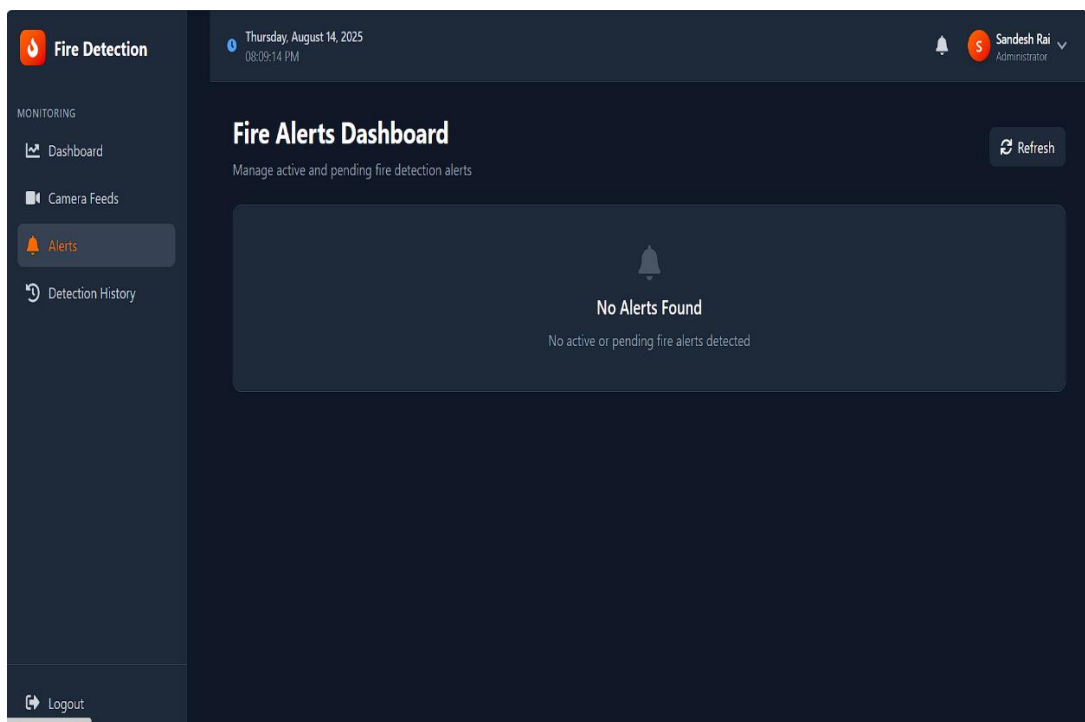
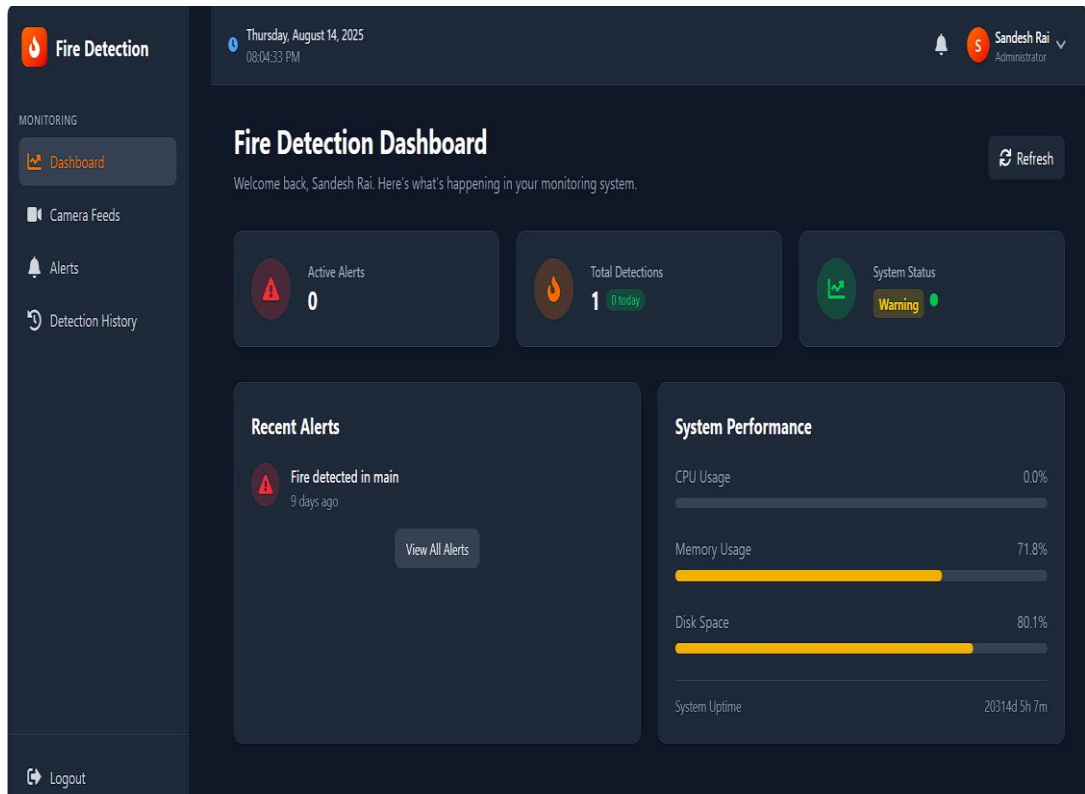
The Fire Detection System can be improved by implementing AI-based predictive models to anticipate potential fire hazards before ignition, reducing risks and improving response times. Developing a mobile application will allow Admin users to monitor live feeds and receive alerts in real time, even when they are away from the site. Cloud-based storage of alert logs and video feeds will help in long-term analysis, reporting, and better decision-making. Adding role-based features for Admin users, such as customizable alert permissions, camera management, and secure access control, will enhance accountability and system management.

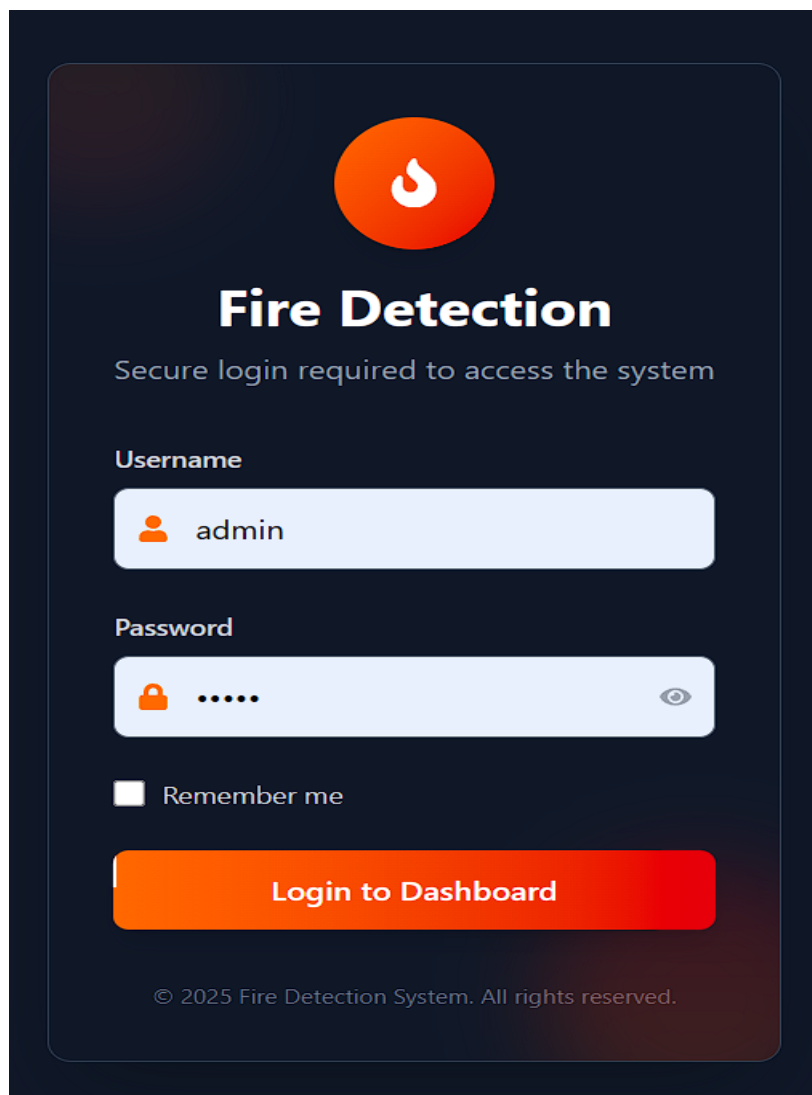
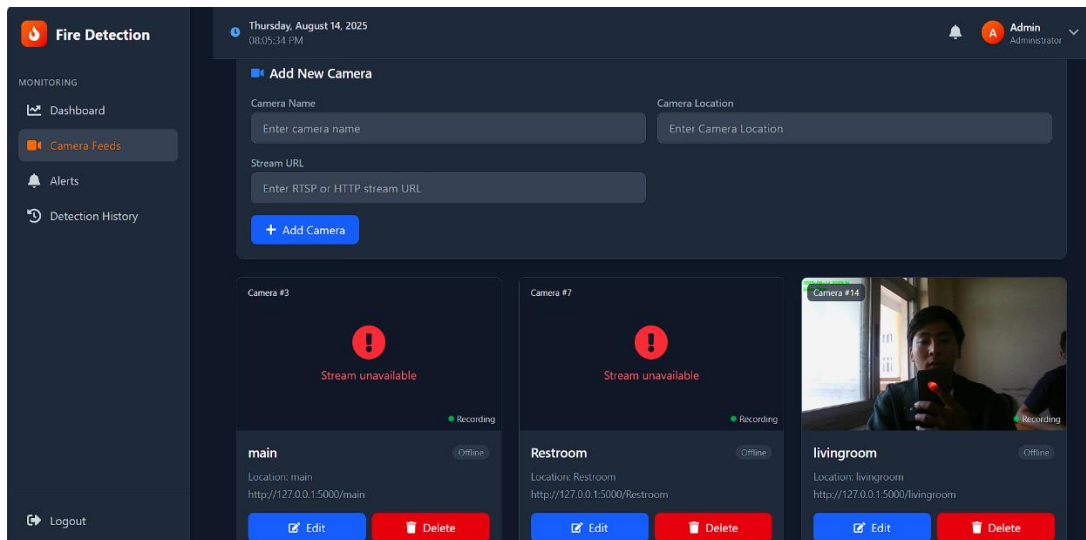
Expanding the system's scalability will allow it to cover multiple buildings or campuses efficiently, making it suitable for larger institutions. Optimizing energy usage with low-power devices and battery backup will ensure continuous operation during power outages. Integration with IoT devices, such as smoke detectors and sprinklers, will enable fully automated fire response for improved safety. Continuous updates and AI model improvements can help reduce false alarms and increase detection accuracy. These future enhancements will make the Fire Detection System more intelligent, reliable, and adaptable, providing better protection for lives and property in diverse environments.

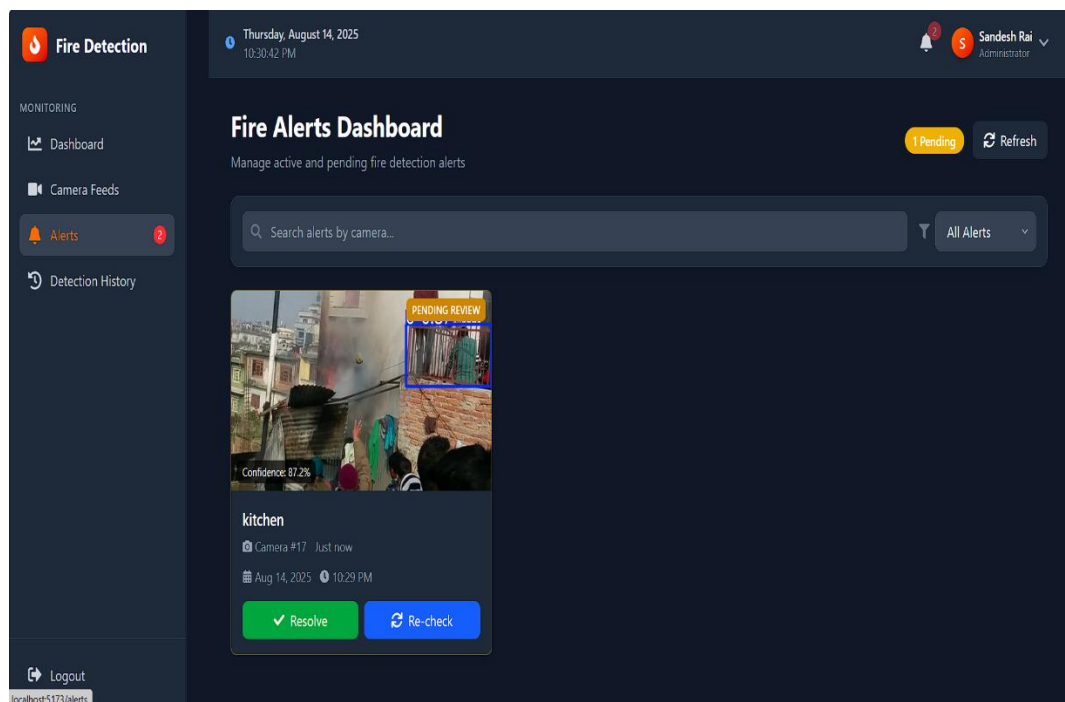
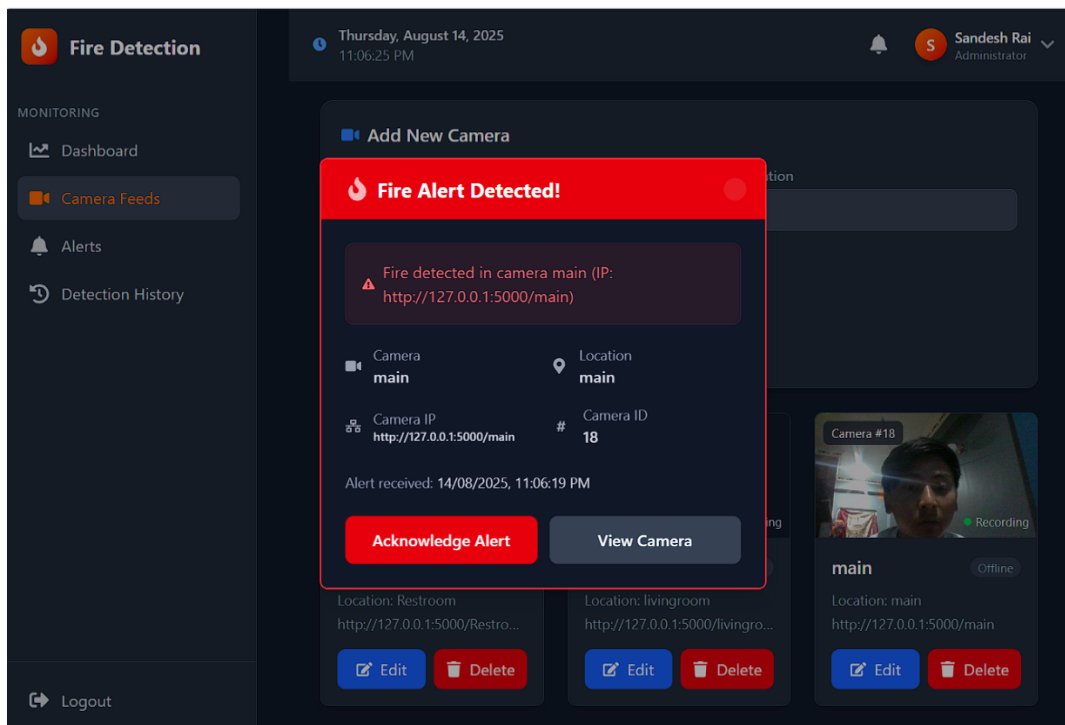
REFERENCES

- [1] M.Sharma and S.Singh, "Overview of Fire Detection Systems: Technologies and Applications," *International Journal of Advance Research in Computer Science*, pp. 102-108, Jun 2017.
- [2] A. T. Rahman, M. F. Rahman, and M. R. Islam, "A Study on Fire Detection and Alarm System," *International Journal of Computer Applications*, pp. 15-20, July 2017.
- [3] D. Drysdale, *An Introduction to Fire Dynamics*, Wiley, 2011.
- [4] Z. Xue, L. Kong, H. Wu and J. Chen, "Real-Time Fire Detection Based on RTSP Video Streams Using YOLOv11," *IEEE Access*, p. 10, 2022.
- [5] M. G. a. A. Joshi, "Enhancing Fire Detection Systems Using Deep Learning and Real-Time Video Analysis," in *Proceedings of the International Conference on Image Processing*, New York, NY, USA, 2021.
- [6] N. F. P. A. (NFPA), "Fire Detection Systems," [Online]. Available: www.nfpa.org. [Accessed 13 August 2025].

APPENDIX







 Fire Detection

MONITORING

 Dashboard

 Camera Feeds

 Alerts

1

 Detection History

Thursday, August 14, 2025
11:07:24 PM

 1

 Sandesh Rai
Administrator

 Detection History

Complete history of fire detections from all cameras

 Refresh

 Export

 Search by camera

 All Cameras

 All Statuses

 5 per page

DATE & TIME	CAMERA	LOCATION	CONFIDENCE	PREVIEW	STATUS	ACTION
Aug 14, 2025 11:06 PM Just now	main  Camera #18	Unknown Location	80.1%		active	 Download

Showing 1 of 1 results on page 1 of 1

 Previous

 Next

29